

Provenance, Timestamp Evidence, and Source-Chain Auditability

A Generated Snapshot Method for AI-Assisted Research Programs

Author: Micah Blumberg

Self Aware Networks Research Institute

<https://selfawarenetworks.com>

July 18, 2026

Abstract

AI-assisted research programs accumulate heterogeneous evidence: chats, drafts, public articles, archive captures, code, proof packets, formal specifications, DOI deposits, and mirrors. This paper presents a generated provenance method that keeps those artifact classes separate and freezes provider state at a declared timestamp. The case study is a bounded 41-record revision-audit cohort captured through 2026-07-11; later deposits are outside this snapshot. All 41 captured Zenodo PDFs match their local canonical PDFs in byte size and SHA-256 digest; the captured OSF files have matching byte sizes. The Branch C proof root is counted under explicit root-versus-recursive rules and linked to the exact-scope BC-0550 checkpoint. The method publishes a path-free manifest while retaining a separate private absolute-path manifest for local recovery. Provenance supports chronology, integrity, reproducibility, and navigation; it does not establish theorem truth, empirical validity, priority, authorship consent, or peer review.

Scope and evidence boundary

Snapshot date: 2026-07-18

1. Contribution and Boundary

The contribution is an executable release process, not a claim that timestamps prove a theory. Each release binds a manuscript to machine-readable manifests, count semantics, provider-state assertions, content hashes, and validator output. A record is included because it exists in the governed corpus, not because its scientific claims are accepted.

The controlling boundary is: **provenance is evidence about artifacts; it is not proof of the claims inside those artifacts.**

2. Artifact and Date Taxonomy

Artifact	Supports	Does not establish by itself
Chat export	Workflow, source leads, draft intent, internal chronology.	Scientific truth, public disclosure, theorem proof.
Local manuscript	Development state and exact claim language.	Public availability, independent validation, peer review.
Public article or archive capture	Public presentation or third-party archival availability at a date.	Original composition date, correctness, influence, or intent.

Artifact	Supports	Does not establish by itself
DOI record	Provider metadata, citable identity, version relations, public availability.	Acceptance, proof, or empirical confirmation.
Proof packet or formal specification	The exact statement proved or checked under its assumptions.	Any wider theorem, physical lift, or interpretation excluded by its claim signature.
Script and run log	Reproduction of a computation under recorded inputs and environment.	General theorem truth or behavior outside that run.
Mirror	Redundancy and provider-specific availability.	Binary parity unless hashes or provider checks are recorded.

Composition, public-article, archive-capture, DOI-deposit, mirror, and computation-run dates remain separate fields. A chronology may display several for one object, but it must not collapse them into a single unqualified publication date.

3. Snapshot Construction

The generator reads a bounded corpus map and an independently captured integrity table. It refuses a release build while any version dossier lacks a finite verdict. For each public record it emits provider identifiers, publication date, family, decision classification, and SHA-256. The private manifest adds local source, PDF, snapshot, and approval-gate paths. Public and private manifests are intentionally different products.

The release checks four independent conditions: Zenodo artifact hash, Zenodo artifact size, OSF byte-size parity, and local-PDF hash parity. Byte-size parity is weaker than a cryptographic remote hash and is labeled accordingly.

4. Corpus Snapshot

- Titled public Zenodo works: **41**
- Local PDFs with matching byte size and SHA-256 digest: **41 / 41**
- Captured OSF files with Zenodo byte-size parity: **41 / 41**
- Broken pre-revision evidence paths: **0**
- Historical paths explicitly documented as absent: **8**

Family	Records
branch-c	10
convergence-trilemma	1
established	14
program-16	16

4.1 Public machine-readable record ledger

The complete 41-row ledger is supplied in generated/manifest-public.json and generated/manifest-public.csv, which are the row-level public data products. Keeping the ledger outside the core manuscript preserves machine readability and avoids compressing the evidence table below a practical reading size. Private OSF identifiers and all machine-local paths remain excluded from the public projection.

5. Branch C Count Semantics and Formal Boundary

Counts are not described as live facts. They are snapshot outputs tied to roots, globs, and recursion rules:

Metric	Root and rule	Count
Root BC Markdown	Branch C root, nonrecursive, BC-*.md	218
Recursive BC Markdown	Branch C root, recursive, BC-*.md	239
Root Python checks	Branch C root, nonrecursive, bc*.py	9
Claim signatures, Markdown	Branch C root, recursive, exact filename	276
Claim signatures, JSON	Branch C root, recursive, exact filename	315
Compatibility edges, JSONL	Branch C root, recursive, exact filename	276
Branch directories	Branch C root, immediate directories	315
Lean formal specs	Formal-spec root, nonrecursive, bc*.lean	276

BC-0550 proves the direct certified mediator constructor premise and the arbitrary-endpoint listed-mediator target contract for the finite Farey route. It does **not** prove finite C-22, an infinite HSIT operator theorem, an SIT / Riemann bridge, the Riemann Hypothesis, or physical SIT. The manifest hashes the packet, claim signature, and Lean source so that this boundary can be checked against exact objects.

6. Provider State and Known Discrepancies

Zenodo record 21234562 is publicly resolvable while its OSF mirror remains private and the recorded release workflow says coauthor approval was required. The public projection withholds the private OSF identifier. This is a provider / workflow discrepancy, not evidence that approval occurred. It requires author reconciliation before further mirroring or revision publication.

GitHub binary parity is not asserted. A local staging drop is documented, but a staged path is not a public repository state. Future releases must record a public commit and content hash before using the word mirrored.

7. Public and Private Manifest Layers

The public manifest contains DOI / provider identifiers, titles, dates, classifications, and content hashes. The private manifest adds machine-local absolute paths and approval gates. The private file is excluded from a public deposit unless every path is intentionally disclosed. This split preserves local restart capability without publishing usernames, temporary directories, private source containers, or unpublished routes.

8. Reproducibility and Validation

A valid release includes manifest-public.json, manifest-public.csv, the private recovery manifest, validator output, generated LaTeX tables, and release-hashes.json. The build fails on an incomplete dossier set, a corpus count other than 41, a missing required checkpoint, or a parity failure. The generated files are treated as the factual source for the manuscript, and prose counts are regenerated rather than hand-edited.

9. Limitations

- Matching byte size and SHA-256 digest is strong fixity evidence, not a mathematical proof of identity and not evidence of scientific correctness.
- OSF byte-size parity is weaker than direct remote cryptographic-hash parity.
- Provider metadata can be wrong or inconsistent with human approval workflows.
- Local filesystem timestamps are weaker chronology evidence than independently archived public records.
- AI-generated source maps can locate candidates but require independent source verification.
- A formal proof object establishes only its exact checked statement and assumptions.

10. Conclusion

An auditable research program needs more than a list of links. It needs explicit artifact classes, noncollapsed date types, separate public / private path layers, frozen provider state, deterministic counts, content hashes, exact proof boundaries, and validators that fail closed. This generated snapshot supplies that structure for a bounded 41-record SIT / SAN / HSIT revision-audit cohort while preserving the decisive rule: provenance can make a claim traceable, but cannot make it true.

Approval and Release Gate

This is a private Version 2 correction candidate. No Zenodo version, OSF change, GitHub push, or public priority statement is authorized by this build. Any later public release must contain only the approved path-safe artifacts; private absolute-path manifests remain excluded. The snapshot preserves provider state captured through 2026-07-11, and its provenance evidence does not establish scientific truth, priority, peer review, coauthor consent for other works, or any theorem beyond the exact formal-artifact boundaries stated here.

References

1. W3C. *PROV-DM: The PROV Data Model*. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-dm/>
2. W3C. *PROV-O: The PROV Ontology*. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-o/>
3. DataCite Metadata Working Group. *DataCite Metadata Schema*. <https://schema.datacite.org/>
4. Data Citation Synthesis Group. *Joint Declaration of Data Citation Principles*. FORCE11, 2014. doi:10.25490/a97f-egykh
5. Wilkinson, M. D., et al. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* 3, 160018 (2016). doi:10.1038/sdata.2016.18
6. Zenodo. *Manage versions*. <https://help.zenodo.org/docs/deposit/manage-versions/>
7. Di Cosmo, R. Archiving and Referencing Source Code with Software Heritage. In *ICMS 2020*, 362-373. doi:10.1007/978-3-030-52200-1_36
8. Citation File Format. *CITATION.cff documentation*. <https://citation-file-format.github.io/>